

Ниже представлены методические указания к лабораторной работе по теме «Основы объектно-ориентированного программирования в C++», охватывающие все разделы приложенных лекционных материалов. Документ содержит теоретическую часть с авторскими примерами, общее задание, 25 индивидуальных вариантов, контрольные вопросы и справочные приложения.

Методические указания к лабораторной работе

«Основы объектно-ориентированного программирования в C++ (классы, конструкторы, статические члены)»

1. Цель работы

Изучение и практическое применение основных возможностей объектно-ориентированного программирования в языке C++:

- создание классов, инкапсуляция данных (`private` – поля, `public` – методы);
- конструкторы (по умолчанию, с параметрами, копирования, делегирующие, преобразования);
- списки инициализации членов;
- деструкторы;
- статические члены класса (поля и методы);
- указатель `this` и цепочка вызовов методов;
- вложенные типы (`enum`, `using`);
- разделение объявления и реализации (заголовочные файлы);
- использование контейнеров STL (`vector`, `list`) и ввод/вывод.

Работа рассчитана на **4 академических часа**.

2. Основные соглашения и требования

- Имена **нестатических** полей класса должны начинаться с префикса `m_` (например `m_name`).
- Имена **статических** полей класса должны начинаться с префикса `s_` (например `s_counter`).
- Все поля класса должны быть **закрытыми** (`private`). Доступ к ним осуществляется через **геттеры** и **сеттеры** (методы получения/изменения значений).
- При объявлении переменных обязательно использовать **инициализаторы** (списочную или прямую инициализацию).
- Фигурные скобки, открывающие тело функции или конструктора, размещаются на **новой строке**.
- Динамическое выделение памяти (`new/delete`) **не используется** – все объекты создаются в автоматической памяти или в контейнерах STL.

Теоретическая часть (краткое изложение с примерами)

2.1. Конструкторы и списки инициализации

Конструктор – специальная функция-член, автоматически вызываемая при создании объекта.

Список инициализации позволяет инициализировать поля до выполнения тела конструктора.

Пример 1. Класс Book

Поля: `m_title` (строка), `m_pages` (целое), `m_price` (вещественное).

```
#include <iostream>
#include <string>

class Book
{
private:
    std::string m_title{ "Unknown" };    // инициализатор по умолчанию
    int         m_pages{ 0 };
    double      m_price{ 0.0 };

public:
    // Конструктор с параметрами (использует список инициализации)
    Book(const std::string& title, int pages, double price)
        : m_title{ title }
        , m_pages{ pages }
        , m_price{ price }
    {
        std::cout << "Book '" << m_title << "' created.\n";
    }

    // Конструктор по умолчанию (явно заданный)
    Book() = default;

    // Геттеры
    const std::string& getTitle() const { return m_title; }
    int getPages() const { return m_pages; }
    double getPrice() const { return m_price; }

    // Сеттер для цены (пример)
    void setPrice(double price) { m_price = price; }
};
```

Демонстрация:

```
int main()
{
    Book b1;                                // конструктор по умолчанию
    Book b2{ "C++ Guide", 500, 29.99 };     // конструктор с параметрами

    std::cout << b2.getTitle() << " (" << b2.getPages() << " p.) - "
              << b2.getPrice() << " $\n";

    return 0;
}
```

2.2. Делегирование конструкторов

Конструктор может вызывать другой конструктор того же класса из списка инициализации.

Пример 2. Класс `Employee`

Поля: `m_name`, `m_id`, `m_hourlyRate`.

```
class Employee
{
private:
    std::string m_name;
    int         m_id;
    double      m_hourlyRate;

public:
    // Основной конструктор
    Employee(const std::string& name, int id, double rate)
        : m_name{ name }
        , m_id{ id }
        , m_hourlyRate{ rate }
    {}

    // Делегирующий конструктор (id и rate по умолчанию)
    Employee(const std::string& name)
        : Employee{ name, 0, 10.0 } // делегирование
    {}
};
```

2.3. Конструктор копирования

Создаёт объект как копию другого объекта того же класса.

Если не определён пользователем, компилятор генерирует **неявный конструктор копирования**, выполняющий поэлементное копирование.

Пример 3. Явный конструктор копирования с диагностикой

```
class CopyTracker
{
private:
    int m_value;
public:
    CopyTracker(int val) : m_value{ val } {}
    CopyTracker(const CopyTracker& other)
        : m_value{ other.m_value }
    {
        std::cout << "Copy constructor called\n";
    }
    int getValue() const { return m_value; }
};
```

Запрет копирования `c = delete;`

```
class NoCopy
{
public:
    NoCopy() = default;
    NoCopy(const NoCopy&) = delete;
};
```

2.4. Конструкторы преобразования и ключевое слово `explicit`

Конструктор, который можно вызвать с одним аргументом, может служить для неявного преобразования типа. Ключевое слово `explicit` запрещает такое неявное преобразование.

Пример 4. Класс `Temperature`

```
class Temperature
{
private:
    double m_celsius;
public:
    explicit Temperature(double celsius) // явный конструктор
        : m_celsius{ celsius }
    {}
    double getCelsius() const { return m_celsius; }
};

void printTemperature(const Temperature& t)
{
    std::cout << t.getCelsius() << " °C\n";
}

int main()
{
    // printTemperature(36.6); // ОШИБКА: конструктор explicit, неявное
    // преобразование запрещено
    printTemperature(Temperature{ 36.6 }); // ОК: явное создание объекта
}
```

2.5. Деструктор

Деструктор вызывается автоматически при уничтожении объекта. Имеет имя `~ИмяКласса`, не принимает аргументов, не возвращает значения.

Пример 5. Класс `Logger` – записывает в консоль факт создания и уничтожения.

```
class Logger
{
private:
    static inline int s_counter{ 0 };
```

```

    int m_id;
public:
    Logger() : m_id{ ++s_counter }
    {
        std::cout << "Logger " << m_id << " created\n";
    }
    ~Logger()
    {
        std::cout << "Logger " << m_id << " destroyed\n";
    }
};

```

2.6. Статические члены класса

Статическое поле принадлежит самому классу, а не отдельным объектам. **Статический метод** может обращаться только к статическим полям (у него нет указателя `this`).

Пример 6. Генератор уникальных идентификаторов

```

class IDGenerator
{
private:
    static inline int s_nextID{ 1000 }; // инициализация внутри класса (C++17)
public:
    static int getNextID() { return s_nextID++; }
};

int main()
{
    std::cout << IDGenerator::getNextID() << '\n'; // 1000
    std::cout << IDGenerator::getNextID() << '\n'; // 1001
}

```

2.7. Указатель `this` и цепочка методов

Внутри нестатического метода доступен указатель `this` – адрес текущего объекта. Возвращая `*this` по ссылке, можно организовывать цепочки вызовов.

Пример 7. Класс `Calculator`

```

class Calculator
{
private:
    double m_value{ 0.0 };
public:
    Calculator& add(double x) { m_value += x; return *this; }
    Calculator& sub(double x) { m_value -= x; return *this; }
    Calculator& mult(double x) { m_value *= x; return *this; }
    double getValue() const { return m_value; }
}

```

```
};

int main()
{
    Calculator calc;
    calc.add(5).sub(2).mult(3);    // цепочка вызовов
    std::cout << calc.getValue(); // (0+5-2)*3 = 9
}
```

2.8. Вложенные типы (типы-члены)

Можно определять перечисления (**enum**) или псевдонимы типов (**using**) внутри класса для улучшения структуры кода.

Пример 8. Класс **Car** с вложенным перечислением **Type**

```
class Car
{
public:
    enum class Type { SEDAN, SUV, HATCHBACK };
private:
    Type m_type;
    std::string m_model;
public:
    Car(const std::string& model, Type t)
        : m_model{ model }, m_type{ t }
    {}
    Type getType() const { return m_type; }
};

int main()
{
    Car myCar{ "Focus", Car::Type::HATCHBACK };
}
```

2.9. Разделение объявления и реализации (заголовочные файлы)

Классы обычно помещают в заголовочный файл (**.h**), а определение методов – в файл реализации (**.cpp**). Простые методы (геттеры) можно оставить в заголовке (они неявно **inline**).

Пример 9. Класс **Point**

Point.h

```
#ifndef POINT_H
#define POINT_H

class Point
{
```

```
private:
    double m_x{}, m_y{};
public:
    Point(double x = 0, double y = 0);
    double getX() const { return m_x; }
    double getY() const { return m_y; }
    double distanceTo(const Point& other) const;
};

#endif
```

Point.cpp

```
#include "Point.h"
#include <cmath>

Point::Point(double x, double y) : m_x{ x }, m_y{ y } {}

double Point::distanceTo(const Point& other) const
{
    double dx = m_x - other.m_x;
    double dy = m_y - other.m_y;
    return std::sqrt(dx*dx + dy*dy);
}
```

2.10. Контейнеры STL: `vector` и `list`

В лабораторной работе для хранения наборов объектов используются контейнеры `std::vector` и `std::list`. Основные операции:

- добавление элемента в конец: `push_back(значение)`
- получение размера: `size()`
- доступ по индексу: `operator[]` (только для `vector`)
- итерация: `begin()`, `end()` (поддерживается range-based for)
- вставка/удаление: `insert()`, `erase()` (для `list` эффективнее)

Пример 10. Хранение объектов `Book` в векторе

```
#include <vector>
#include "Book.h"

int main()
{
    std::vector<Book> library;
    library.push_back(Book{ "1984", 328, 12.99 });
    library.push_back(Book{ "Brave New World", 288, 14.50 });

    for (const auto& book : library)
```

```
std::cout << book.getTitle() << '\n';  
}
```

3. Общее задание для студентов

Необходимо разработать консольную программу на C++, состоящую из **трёх или четырёх взаимосвязанных классов**, которая выполняет следующие действия:

1. **Ввод данных** – с клавиатуры или из заранее подготовленного массива (константы). Данные могут представлять собой набор сущностей (студенты, книги, товары, сотрудники и т.п.).
2. **Хранение данных** – с использованием контейнеров `std::vector` или `std::list`. Каждый элемент контейнера – объект одного из разработанных классов.
3. **Вычислительная часть** – обработка данных в соответствии с вариантом (подсчёт сумм, средних, поиск, фильтрация, сортировка и т.п.).
4. **Форматированный вывод** – результаты вычислений и исходные данные выводятся на экран в удобочитаемом виде (таблицы, заголовки, разделители).

Требования к реализации классов:

- Все поля **private**, имена с префиксами `m_` (нестатические) и `s_` (статические).
- Для доступа к полям использовать **геттеры** (возвращают значения или константные ссылки) и, при необходимости, **сеттеры**.
- Обязательно наличие **конструкторов**: по умолчанию (если нужен), с параметрами, копирования (если не запрещён явно).
- В конструкторах применять **списки инициализации**.
- Если в задании требуется подсчёт количества созданных объектов или генерация идентификаторов – использовать **статические поля и методы**.
- При необходимости можно определить **деструктор** для логирования, если это оправдано.
- Для улучшения структуры использовать **вложенные типы** (например, перечисления для категорий).
- В методах, где объект не изменяется, указывать спецификатор `const`.
- Для цепочек операций (например, добавление данных) можно применять возврат `*this`.

Запрещено использование явного динамического выделения памяти (`new/delete`). Все объекты создаются либо в автоматической памяти, либо в контейнерах STL.

4. Варианты индивидуальных заданий

Ниже приведены описания 25 вариантов. В каждом варианте указаны классы, их краткое назначение и дополнительные требования.

Вариант 1. Учёт студентов

Классы: `Student` (имя, курс, оценки), `Group` (название, список студентов), `Deanery` (список групп).

Задание: Ввод данных о студентах (можно задать константно). Для каждой группы вычислить средний балл, вывести список отличников (все оценки ≥ 9). Использовать `vector`. Вывод в виде таблицы.

Вариант 2. Библиотека

Классы: **Book** (название, автор, год, количество экземпляров), **Reader** (ФИО, номер читательского билета), **Library** (список книг, список читателей).

Задание: Имитировать выдачу книг: читатель берёт книгу – уменьшается количество экземпляров. Вывести список книг, которые сейчас на руках (кол-во экземпляров < начального). Использовать **list** для книг.

Вариант 3. Интернет-магазин

Классы: **Product** (название, цена, рейтинг), **Category** (название, список продуктов), **Cart** (список выбранных продуктов).

Задание: Сформировать корзину из нескольких продуктов, вычислить общую стоимость. Вывести продукты с рейтингом выше 4.0, отсортированные по цене. Использовать **vector**.

Вариант 4. Банковские счета

Классы: **Client** (ФИО, паспорт), **Account** (номер счёта, баланс, владелец), **Bank** (список счетов).

Задание: Создать несколько счетов. Выполнить перевод средств между двумя счетами (учесть, что счёт не может уйти в минус). Вывести все счета с балансом после операций. Использовать **vector**.

Вариант 5. Спортивные команды

Классы: **Player** (имя, возраст, результативность – голы), **Team** (название, список игроков), **League** (список команд).

Задание: Найти команду с максимальной суммарной результативностью, вывести её состав, отсортированный по возрасту. Использовать **list** для игроков.

Вариант 6. Транспортное агентство

Классы: **Vehicle** (марка, год выпуска, стоимость аренды в день), **Client** (имя, телефон), **Rental** (клиент, транспорт, количество дней).

Задание: Сформировать несколько аренд. Вычислить общую выручку. Вывести список клиентов, арендовавших автомобили старше 5 лет. Использовать **vector**.

Вариант 7. Геометрические фигуры

Классы: **Point** (x, y), **Line** (две точки), **Polygon** (список точек – вершин).

Задание: Создать многоугольник (задать вершины). Вычислить его периметр (сумма длин всех сторон). Использовать **vector<Point>**. Метод длины отрезка – **const**.

Вариант 8. Учёт сотрудников

Классы: **Employee** (имя, должность, оклад), **Department** (название, список сотрудников), **Company** (список отделов).

Задание: Найти отдел с максимальной средней зарплатой, вывести сотрудников этого отдела, упорядоченных по окладу. Использовать **vector**.

Вариант 9. Зоопарк

Классы: `Animal` (вид, кличка, возраст), `Enclosure` (номер, список животных), `Zoo` (список вольеров).

Задание: Определить, сколько животных каждого вида (например, "лев", "слон") содержится в зоопарке. Вывести статистику по видам. Использовать `list`.

Вариант 10. Авиабилеты

Классы: `Flight` (номер рейса, пункт назначения, время вылета), `Passenger` (ФИО, номер паспорта), `Booking` (рейс, пассажир).

Задание: Создать список бронирований. Вывести все рейсы, на которые зарегистрировано более 3 пассажиров. Использовать `vector<Booking>`.

Вариант 11. Склад

Классы: `Item` (наименование, артикул, цена), `Warehouse` (название, список товаров с остатками – можно вложенный класс `Stock`).

Задание: Вывести товары, количество которых меньше заданного порога (например, 5). Использовать `vector`.

Вариант 12. Поликлиника

Классы: `Doctor` (ФИО, специальность), `Patient` (ФИО, возраст), `Appointment` (доктор, пациент, дата).

Задание: Для каждого врача вывести список его пациентов (уникальных). Использовать `list` и вспомогательные контейнеры.

Вариант 13. Оценки студентов (другая модель)

Классы: `Subject` (название), `Student` (имя, список оценок – можно `map<Subject, int>`), `Journal` (список студентов).

Задание: Вычислить средний балл по каждому предмету. Вывести предметы, где средний балл ниже 4.0.

Вариант 14. Гостиница

Классы: `Room` (номер, категория, цена за ночь), `Guest` (имя, паспорт), `Reservation` (гость, номер, количество ночей).

Задание: Подсчитать общую выручку от всех бронирований. Вывести список гостей, проживающих в номерах категории "люкс".

Вариант 15. Футбольный чемпионат

Классы: `Team` (название, город), `Match` (команда-хозяин, команда-гость, счёт), `Tournament` (список команд, список матчей).

Задание: По результатам матчей сформировать турнирную таблицу (очки: победа – 3, ничья – 1). Вывести таблицу, отсортированную по очкам.

Вариант 16. Магазин электроники

Классы: `Device` (тип – смартфон/ноутбук, модель, цена), `Customer` (имя, бюджет), `Purchase` (покупатель, устройство).

Задание: Сымитировать покупки: каждый покупатель покупает устройство, если хватает бюджета. Вывести список покупок и оставшиеся бюджеты.

Вариант 17. Кинотеатр

Классы: **Movie** (название, жанр, длительность), **Session** (фильм, время, зал), **Ticket** (сеанс, место, цена).

Задание: Для каждого фильма подсчитать общую выручку со всех сеансов. Вывести фильмы, у которых выручка превышает среднюю.

Вариант 18. Туристическое агентство

Классы: **Tour** (страна, длительность, стоимость), **Client** (ФИО, телефон), **Order** (клиент, тур, дата).

Задание: Найти самый популярный тур (по количеству заказов). Вывести список клиентов, заказавших этот тур.

Вариант 19. Компьютерная игра (инвентарь)

Классы: **Item** (название, тип – оружие/броня/зелье, вес), **Player** (имя, уровень, инвентарь – список предметов), **GameWorld** (список игроков).

Задание: Вычислить общий вес инвентаря каждого игрока. Вывести игроков, чей инвентарь легче 10 кг.

Вариант 20. Студенческое общежитие

Классы: **Student** (ФИО, группа), **Room** (номер, этаж, список студентов), **Dormitory** (список комнат).

Задание: Переселить студентов: из одной комнаты в другую (изменение состава). Вывести список студентов, проживающих на указанном этаже.

Вариант 21. Телефонная книга

Классы: **Contact** (имя, телефон, email), **Group** (название, список контактов), **PhoneBook** (список групп).

Задание: Объединить все контакты из всех групп в общий список (без дубликатов по телефону). Вывести объединённый список, отсортированный по имени.

Вариант 22. Книжный магазин (заказы)

Классы: **Book** (название, автор, цена), **Customer** (имя, email), **Order** (покупатель, книга, количество).

Задание: Подсчитать, сколько экземпляров каждой книги заказано. Вывести книги, заказанные более 5 раз, с общей суммой.

Вариант 23. Спортивные секции

Классы: **Coach** (ФИО, стаж), **Section** (название, тренер, список учеников – класс **Student**), **SportsClub** (список секций).

Задание: Найти тренера с максимальным стажем, вывести всех его учеников (из всех секций, которые он ведёт).

Вариант 24. Ресторан

Классы: **Dish** (название, цена, вес), **Menu** (список блюд), **Order** (список блюд, общая стоимость).

Задание: Создать несколько заказов. Вывести топ-3 самых популярных блюд (по частоте заказов). Использовать **vector**.

Вариант 25. Управление проектами

Классы: `Employee` (имя, должность, ставка), `Task` (название, часы, исполнитель), `Project` (название, список задач).

Задание: Для каждого проекта вычислить общую стоимость трудозатрат (часы * ставка). Вывести проекты, стоимость которых превышает 1000.

5. Контрольные вопросы

1. Что такое класс? Чем отличается от структуры?
 2. Какие спецификаторы доступа вы знаете? Для чего используется `private`?
 3. Как называются нестатические поля класса согласно соглашению?
 4. Что такое конструктор? Каковы его особенности?
 5. Какие виды конструкторов существуют?
 6. Для чего используется список инициализации членов?
 7. В каком порядке инициализируются поля в списке инициализации?
 8. Что такое делегирующий конструктор?
 9. Когда компилятор генерирует неявный конструктор по умолчанию?
 10. Как запретить создание копии объекта?
 11. Что делает ключевое слово `explicit`?
 12. Для чего нужен деструктор?
 13. В какой момент вызывается деструктор?
 14. Что такое статическое поле класса? Как оно инициализируется?
 15. Может ли статический метод обращаться к нестатическим полям? Почему?
 16. Что такое указатель `this`?
 17. Как организовать цепочку вызовов методов?
 18. Что такое вложенный тип? Приведите пример.
 19. Как разделить объявление класса и реализацию его методов (файлы `.h` и `.cpp`)?
 20. Когда метод следует объявлять `const`?
 21. Какие контейнеры STL разрешено использовать в работе? Назовите основные операции с `vector`.
 22. В чём разница между `vector` и `list`?
 23. Что такое итератор? Как его получить?
 24. Как перебрать элементы контейнера с помощью `range-based for`?
 25. Что произойдёт, если не определить конструктор копирования для класса с полями-ссылками?
 26. Можно ли вызвать конструктор из тела другого конструктора напрямую? Почему нет?
 27. Для чего применяется `= default` и `= delete`?
 28. Как передать объект в функцию, чтобы избежать копирования?
 29. Что такое агрегатная инициализация? Применима ли она к классам с `private` полями?
 30. В каких случаях компилятор может неявно преобразовывать типы? Как это предотвратить?
 31. Что такое `constexpr` конструктор? Какие требования к нему предъявляются?
 32. Может ли деструктор быть приватным? Зачем?
 33. Что произойдёт при вызове `std::exit()`? Вызовутся ли деструкторы локальных объектов?
 34. В чём отличие статического поля от глобальной переменной?
-

Приложение А. Справочник по синтаксису C++ (основные конструкции)

А.1. Определение класса

```
class ИмяКласса
{
private:
    // поля (обычно private)
    тип m_поле{ инициализатор };
    static тип s_поле;    // только объявление
public:
    // конструкторы
    ИмяКласса() = default;
    ИмяКласса(параметры) : m_поле{ значение } {}

    // методы
    тип метод() const;
};
// определение статического поля (если не inline)
тип ИмяКласса::s_поле{ значение };
```

А.2. Конструкторы

- **Список инициализации** – после двоеточия, перечисляются инициализируемые поля через запятую.
- **Делегирование** – вызов другого конструктора в списке инициализации.
- **Копирования** – `ИмяКласса(const ИмяКласса& other) : m_поле{ other.m_поле } {}`.
- **Явный** – `explicit ИмяКласса(тип) {}`.

А.3. Деструктор

```
~ИмяКласса()
{
    // тело
}
```

А.4. Статические члены

- **Статическое поле** – объявляется с `static`, определяется вне класса (или `inline` внутри).
- **Статический метод** – объявляется с `static`, не имеет `this`, может обращаться только к статическим полям.

А.5. Указатель `this`

Внутри нестатического метода: `this` – константный указатель на текущий объект.

Возврат `*this` для цепочки: `return *this;`.

А.6. Вложенные типы

```
class Outer
{
public:
    enum class Color { RED, GREEN, BLUE };
    using Number = int;
};
// Доступ снаружи: Outer::Color::RED
```

A.7. `const` методы

```
тип метод() const { /* не изменяет поля */ }
```

Приложение Б. Контейнеры STL: `std::vector` и `std::list`

Б.1. `std::vector<T>`

- `push_back(const T& value)` – добавляет элемент в конец.
- `pop_back()` – удаляет последний элемент.
- `size()` – возвращает количество элементов.
- `empty()` – проверяет, пуст ли вектор.
- `operator[](size_t i)` – доступ по индексу (без проверки границ).
- `at(size_t i)` – доступ с проверкой границ (кидает исключение).
- `begin()` / `end()` – итераторы на начало и конец.
- `insert(iterator pos, const T& value)` – вставка в позицию.
- `erase(iterator pos)` – удаление элемента.

Б.2. `std::list<T>` (двусвязный список)

- `push_back()`, `pop_back()`, `push_front()`, `pop_front()`
- `size()`, `empty()`
- `begin()`, `end()`
- `insert()`, `erase()` – более эффективны, чем у `vector` (не требуют сдвига).

Б.3. Пример использования `vector` и `list` с пользовательскими классами

```
std::vector<Book> books;
books.push_back(Book("Title", 100, 9.99));

std::list<Book> bookList;
bookList.push_back(Book("Another", 200, 15.00));

for (const auto& b : bookList)
    std::cout << b.getTitle() << '\n';
```

Приложение В. Ввод/вывод и форматирование

В.1. Стандартные потоки

- `std::cin` – ввод.
- `std::cout` – вывод.
- `std::endl` – перевод строки + сброс буфера.

В.2. Форматирование (заголовков `<iomanip>`)

- `std::setw(n)` – задаёт ширину поля.
- `std::left` / `std::right` – выравнивание.
- `std::fixed` – фиксированная запись чисел.
- `std::setprecision(p)` – количество знаков после запятой.

Пример форматированного вывода таблицы:

```
#include <iomanip>
std::cout << std::left << std::setw(15) << "Название"
          << std::right << std::setw(8) << "Цена" << '\n';
std::cout << std::setw(23) << std::setfill('-') << "" << std::setfill(' ') <<
'\n';
for (const auto& book : books)
    std::cout << std::left << std::setw(15) << book.getTitle()
              << std::right << std::setw(8) << std::fixed << std::setprecision(2)
              << book.getPrice() << '\n';
```

Методические указания составлены таким образом, чтобы студент мог ознакомиться с необходимыми теоретическими сведениями, изучить примеры кода и выполнить индивидуальное задание, применяя полученные знания. Успешной работы!